

1. Portada

- Saluda y preséntate.
- CodeAtlas: convierte el código en un mapa visual e interactivo.
- Formato: 15 min de explicación + 5 min de demo.

2. El problema — entender

- Entender una app nueva cuesta días leyendo código.
- Quien entra nuevo tarda en ser productivo.

3. El problema — mantener

- Ya hay formatos para dibujar apps; lo difícil es mantenerlos.
- Se actualizan a mano y por separado → se desfasan.
- Están dispersos en herramientas distintas.
- CodeAtlas: un origen de verdad, sincronizado y centralizado.

4. De dónde viene la idea

- La IA traduce muy bien humano ↔ máquina.
- Analiza la app y la pasa a un formato que se dibuja.
- CodeAtlas la transforma en esquema visual.
- Caso estrella: onboarding en minutos, no días.

5. No solo para entenderla entera

- No solo ver la app completa de golpe.
- Día a día: antes de tocar código ves cómo funcionan los archivos que cambiarás.
- Sigues el patrón del proyecto, no rompes la estructura.

6. La visión a futuro

- Programas con un agente de IA (p. ej. Claude Code).
- Un skill hace que al escribir código genere/actualice la doc solo.
- Doc y diagrama siempre al día, sin esfuerzo extra.

7. ¿Qué hace CodeAtlas?

- Subes carpeta .md → la interpreta → diagrama interactivo.
- Ves módulos, pantallas, BD y flujos conectados.
- No es estático: mueves nodos, filtras, ves detalles.

8. Lenguajes y tecnologías

- JavaScript de punta a punta (los que mejor conozco).
- Front: Vue 3 + Vite + Pinia + Vue Flow.
- Back: Node + Express, MySQL, Google Gemini.

9. Arquitectura modular

- Cada área = su propia carpeta.
- Back: routes → controller → service → repository.
- Front: views → components → services → stores.
- Fácil de entender, mantener y ampliar.
- Front y back separados, vía API REST.

10. Base de datos

- MySQL, 6 tablas relacionadas.
- users, user_settings, projects, diagrams, bot_sessions, bot_files.
- diagrams guarda el modelo y el layout en JSON.
- Toda consulta filtra por user_id; borrado en cascada.

11. El formato .md

- Lee archivos .md con un formato concreto.
- 7 tipos de archivo; cada uno = un nodo.
- Frontmatter (IDs) → las flechas del diagrama.
- Secciones ## → el contenido del nodo.

12. Anatomía de un archivo

- Ejemplo: un módulo de backend.
- Campos con IDs (database, depends-on) → flechas.
- Secciones ## Purpose / ## Functions → panel de detalle.

13. El mismo formato, otras piezas

- Pasa rápido: índice, pantalla, módulo frontend, tabla DBML.
- Mismo formato aplicado a piezas distintas.

14. El flujo

- El más especial: una secuencia de pasos entre capas.
- Cada paso lleva un prefijo [capa:módulo/archivo/función].
- Se ilumina el recorrido: pantalla → frontend → backend → BD.
- Ves qué pasa cuando el usuario hace algo, sin leer código.

15. El pipeline del parser

- Al subir los archivos se ejecuta un pipeline lineal.
- Extraer → validar → construir modelo → resolver → layout.
- Resultado: un JSON que dibuja Vue Flow.
- Se guarda modelo + posiciones (reabres como lo dejaste).

16. El asistente de IA

- Describe la app en lenguaje natural; Gemini genera los .md.
- Conversa, guarda historial y valida los archivos.
- Los descargas en un .zip listo para usar.
- Eliges modelo; si se agota la cuota, ofrece cambiar. API key solo en server.
- Cubre el requisito de integrar IA.

17. Seguridad

- JWT (token firmado que caduca).
- Contraseñas con bcrypt, nunca en texto plano.
- Aislamiento: toda consulta filtra por user_id.
- Validación + consultas parametrizadas (anti SQL-injection).

18. Desplegada en producción

- Docker + Kubernetes.
- Servidor interno del instituto: acceso por HTTP (no HTTPS), URL interna.
- Cumple el requisito de despliegue (sin FTP), front/back separados.

19. Próximos pasos

- Analizar apps existentes desde el código (application-diagram).
- Generar código alineado con el diagrama.
- Trabajo en equipo y diagramas compartidos.

20. Cierre / Demo

- Recap en una frase.
- “Y ahora os lo enseño funcionando” → pasa a la demo.

21. DEMO EN VIVO (5 min)

- Login (credenciales listas).
- Dashboard: proyectos y diagramas recientes.
- Asistente IA: describe app → genera .md → descarga zip.
- Generar diagrama: sube .md, muestra progreso.
- Canvas: mover nodos, undo/redo, modo Flujos, clic en nodo, filtros.
- Cierre: vuelve al dashboard y di “Gracias”.
- Plan B: si falla la red/IA, salta al Canvas con un diagrama ya hecho.